UNIVERSITÉ TOULOUSE CAPITOLE

DevOps

Culture, collaboration et automatisation pour livrer en continu

6h de cours magistral | 18h de travaux dirigés

Harold Parpex

ORGANISATION DU COURS

Modalités d'évaluation et volume horaire

Évaluation

20 % Comportement, participation, ponctualité

40 % Deux rendus de travaux pratiques

40 % Évaluation sur table

Volume horaire

6h Cours magistral — 4 séances de 1h30

18h Travaux dirigés — mise en pratique

PLAN DU COURS

Quatre séances, une histoire

01	**Introduction au DevOps**	Pourquoi le DevOps existe, culture, CALMS
02	**Git et collaboration**	Versionner le code, travailler en équipe
03	**CI/CD**	Du git push au déploiement automatique
04	**Conteneurisation et orchestration**	Docker, optimisation d'image, Kubernetes

Fil rouge : TaskFlow, une petite API développée par quatre étudiants, du ZIP au déploiement scalable.

SÉANCE 1 · 1H30

Introduction au DevOps

Comprendre pourquoi le DevOps est apparu, sa culture et ses piliers — avant de toucher un seul outil.

FIL ROUGE — TASKFLOW

Une API, quatre développeurs

Alice, Bob, Chloé et Karim développent **TaskFlow** , une petite API de gestion de tâches.
Aujourd'hui, leur méthode de mise en production :

Un développeur code en local → **Fichier ZIP envoyé par mail** → **Copié à la main sur le serveur de production**

On garde cette histoire en tête tout au long des quatre séances : chaque brique DevOps vient corriger un de ces problèmes.

Avant DevOps : deux mondes séparés

Développement

Veut livrer vite, ajouter des fonctionnalités, changer le code souvent.

Code | Fonctionnalités | Tests

V
S

Opérations

Veut un système stable, veut éviter tout changement risqué.

Serveurs | Réseau | Sécurité | Monitoring

Deux équipes, deux objectifs contradictoires, souvent aucune communication directe.

Les problèmes du modèle traditionnel

Déploiements rares et risqués

Chaque mise en production est un événement redouté.

Silos et « jeu du blâme »

Quand ça casse, on cherche un coupable plutôt qu'une cause.

Tâches manuelles répétitives

Installation, configuration et tests refaits à la main chaque fois.

Délais de livraison longs

Des semaines entre une idée et sa mise en production.

Sécurité ajoutée trop tard

Un audit de sécurité juste avant la mise en production, trop tard pour corriger sans retarder la livraison.

Vulnérabilités découvertes en production

Sans scan continu des dépendances, les failles sont trouvées trop tard — c'est le problème que résout le DevSecOps.

Historique : Agile → DevOps → Cloud

Le mouvement DevOps ne sort pas de nulle part : chaque décennie ajoute une brique sur laquelle la suivante s'appuie.

2001

Manifeste Agile

Itérations courtes côté développement.

2009

Premier « DevOpsDays »

L'agilité s'étend aux opérations.

2013

Docker

La conteneurisation se démocratise.

2014

Kubernetes

Google ouvre son orchestrateur de containers.

2015+

Cloud natif

Infrastructure programmable, GitOps, DevSecOps.

Le DevOps est une culture qui réunit développement et exploitation pour livrer des logiciels plus vite, plus souvent, et de manière plus fiable.

La culture DevOps

Responsabilité partagée

Ceux qui codent participent aussi à l'exploitation.

Analyse sans blâme

Un incident est une occasion d'apprendre, pas de désigner un coupable.

Amélioration continue

Chaque cycle de livraison sert à ajuster le suivant.

Time to market réduit

Une idée arrive chez l'utilisateur en jours, pas en mois.

Sécurité de la supply chain

La sécurité est intégrée dès le code et les dépendances, pas ajoutée à la fin (DevSecOps).

Collaboration Dev-Sec-Ops

Développement, sécurité et exploitation avancent ensemble, pas en cascade.

CALMS — les cinq piliers

C

Culture

Collaboration entre équipes

A

Automation

Éliminer le travail manuel répétitif

L

Lean

Petits lots, réduire le gaspillage

M

Measurement

Mesurer pour décider

S

Sharing

Partager savoirs et outils

CI/CD en un coup d'œil

Nous détaillerons ces trois notions en séance 3 — pour l'instant, la vue d'ensemble.

Intégration continue (CI)	Chaque changement est testé et assemblé automatiquement.
Livraison continue (CD)	Le logiciel est toujours prêt à être déployé.
Déploiement continu	Chaque changement validé part en production sans action humaine.

Infrastructure as Code

L'infrastructure (serveurs, réseaux, bases de données) est décrite dans des fichiers versionnés — plutôt que configurée à la main, clic par clic.

```
# infrastructure.yaml
server:
type: web
replicas: 3
region: eu-west
```

Résultat : une infrastructure reproductible, versionnée, et revue comme du code.

Monitoring et observabilité

Observabilité

Métriques

CPU, latence, taux d'erreur — des chiffres suivis en continu.

Logs

L'historique détaillé de ce que fait l'application.

Traces

Le parcours d'une requête à travers les différents services.

Trois piliers complémentaires : observer, c'est pouvoir répondre à « pourquoi ce comportement ? », pas seulement « est-ce que ça marche ? ».

Les boucles de rétroaction



Exemple concret : avant et après DevOps

INDICATEUR	AVANT	APRÈS
Fréquence de déploiement	1 fois par mois	**Plusieurs fois par jour**
Temps de récupération après incident	Plusieurs heures	**Quelques minutes**
Taux d'échec des déploiements	Élevé	**Faible**

FIL ROUGE — TASKFLOW

Alice, Bob, Chloé et Karim veulent maintenant travailler ensemble sur le même code, sans s'écraser les uns les autres.

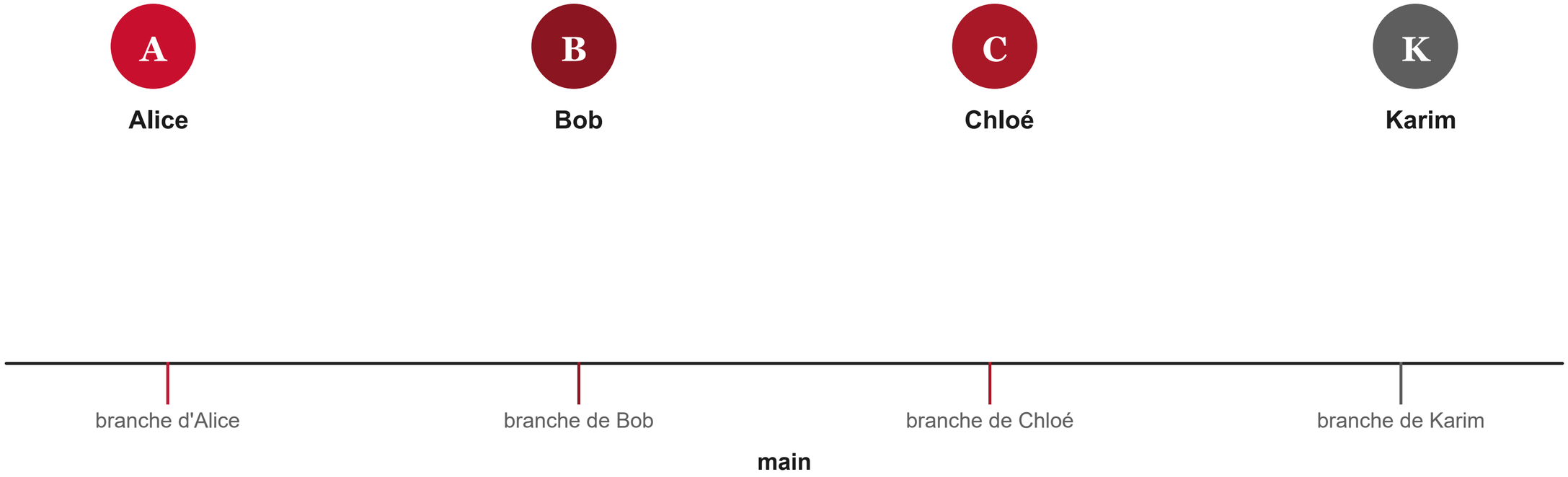
Prochaine étape : versionner le code et collaborer avec Git.

SÉANCE 2 · 1H30

Git et collaboration

Git n'est pas qu'un outil de sauvegarde : c'est la première brique DevOps de collaboration.

Quatre développeurs, un seul dépôt



Chacun travaille sur sa propre branche, puis fusionne dans **main** — le dépôt **Git** partagé, hébergé sur **GitLab**.

Git n'est pas GitHub / GitLab

Git

Un logiciel de contrôle de version, installé en local. Il fonctionne sans connexion internet.

GitHub / GitLab

Des plateformes qui hébergent les dépôts Git en ligne et ajoutent la collaboration : revue de code, tickets, CI/CD.

Le vocabulaire de base

Repository	Le dépôt : tout l'historique du projet.
Commit	Un instantané des changements, avec un message.
Branch	Une ligne de travail indépendante, isolée du reste.
Merge	La fusion d'une branche dans une autre.

Le cycle Git de base

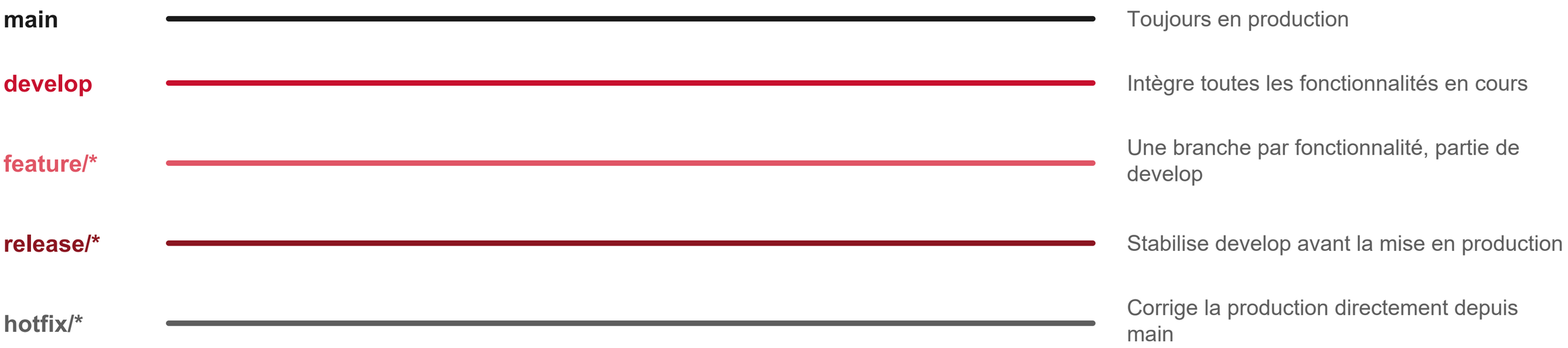


Ce cycle se répète en permanence, souvent plusieurs fois par heure au sein d'une équipe.

Les commandes essentielles

```
$ git clone https://gitlab.com/taskflow/api.git
$ git add .
$ git commit -m "Ajoute la route de création de tâche"
$ git push origin feature/creation-tache
$ git pull origin main
```

GitFlow


main
Toujours en production
develop
Intègre toutes les fonctionnalités en cours
feature/*
Une branche par fonctionnalité, partie de develop
release/*
Stabilise develop avant la mise en production
hotfix/*
Corrige la production directement depuis main

GitFlow face au trunk-based development

GitFlow

Plusieurs branches longues (develop, release, hotfix).
Structuré, mais lourd pour les équipes qui livrent souvent.

Trunk-based

Des branches courtes, fusionnées rapidement dans main.
Adapté au CI/CD et aux déploiements fréquents.

Pull / Merge Requests

Karim crée une branche → **Il ouvre une Merge Request** → **Alice relit et commente** → **Fusion dans main**

La Merge Request est le lieu où le code est discuté avant d'entrer dans le tronc commun.

Pourquoi relire le code d'un autre

Karim ouvre sa Merge Request. Avant qu'elle ne rejoigne main, Alice la relit : elle repère un bug, propose une meilleure façon d'écrire une fonction, et comprend au passage ce que fait cette partie du code — qu'elle n'avait jamais touchée.

C'est tout l'intérêt de la revue : des bugs trouvés plus tôt, une connaissance qui circule dans l'équipe, un style de code qui reste cohérent. Une seule règle : on commente le code, jamais la personne qui l'a écrit.

Bien mener une revue de code

Une convention de code	Nommage, formatage, structure des fichiers — décidés une fois pour toute l'équipe, pour ne plus en discuter dans chaque revue.
Repérer les redondances	Une fonction déjà écrite ailleurs, une logique dupliquée — l'occasion de factoriser avant que ça ne se propage.
S'appuyer sur l'IA	Un premier passage automatisé (style, bugs évidents, suggestions) avant la lecture humaine — qui reste seule décisionnaire.
Des revues courtes et fréquentes	Mieux vaut relire dix petites Merge Requests qu'une seule qui change tout le projet.

Conventional Commits

Un format de message de commit lisible par les humains — et par les outils.

```
feat: ajoute l'endpoint de suppression de tâche

fix: corrige le tri des tâches par date

chore: met à jour les dépendances

docs: complète le README d'installation
```

.gitignore, tags et releases

.gitignore

Exclut du suivi les fichiers qui n'ont rien à faire dans l'historique.

```
node_modules/
.env
*.log
```

Tags & releases

Marquent un point précis de l'historique comme une version publiée.

```
git tag v1.2.0
git push --tags
```

Quelques erreurs classiques

- Committer directement sur `main`
- Messages de commit vagues (« fix », « update »)
- Un `force push` qui écrase le travail des autres
- Un seul commit fourre-tout pour dix fonctionnalités

FIL ROUGE — TASKFLOW

Le code de TaskFlow est maintenant versionné, revu et fusionné en équipe. Mais chaque fusion est encore vérifiée à la main.

Prochaine étape : automatiser les tests et la mise en production avec le CI/CD.

SÉANCE 3 · 1H30

CI/CD

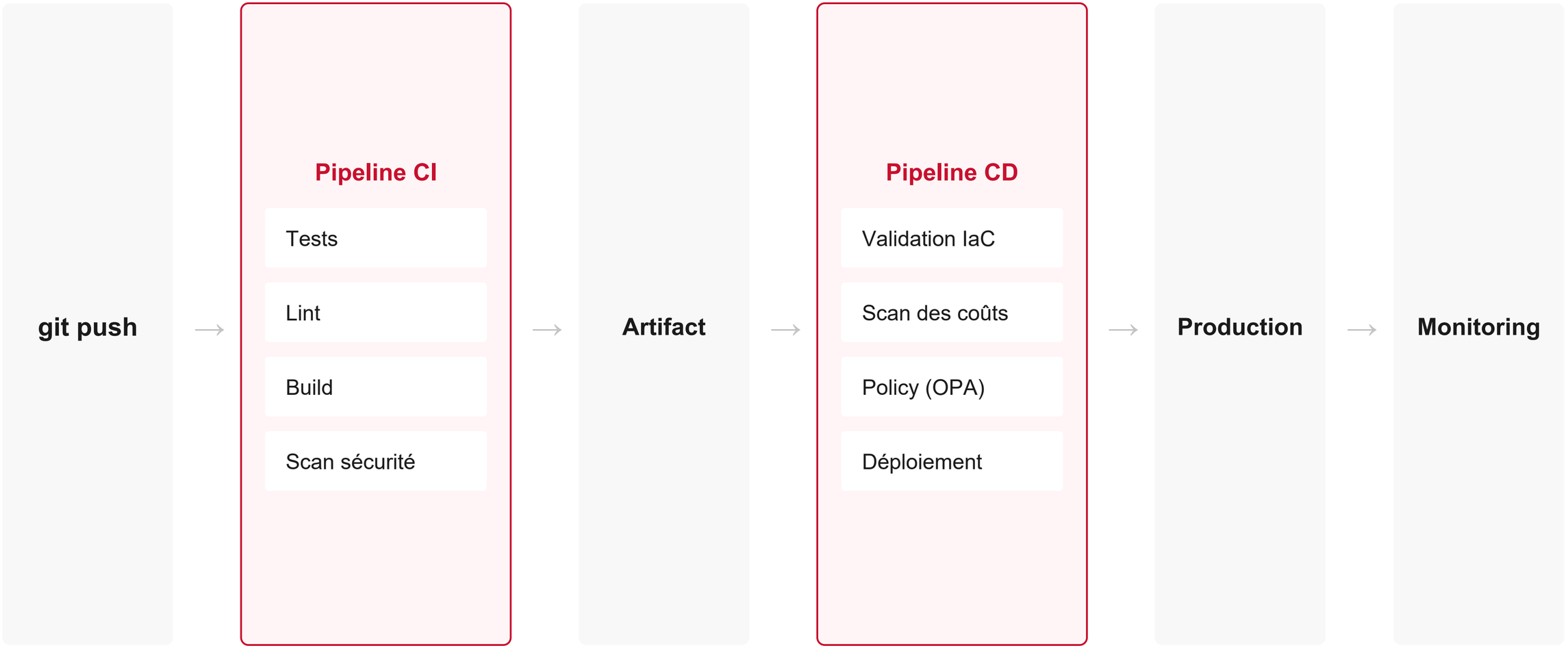
Comment un simple git push peut finir en déploiement en production.

FIL ROUGE — TASKFLOW

Chaque push doit être vérifié

L'équipe fusionne des Merge Requests plusieurs fois par jour. Relire à la main tous les tests avant chaque fusion devient impossible : il faut automatiser.

Le pipeline complet



Un seul push déclenche toute cette chaîne, sans intervention manuelle — la CD porte ses propres contrôles, au même titre que la CI.

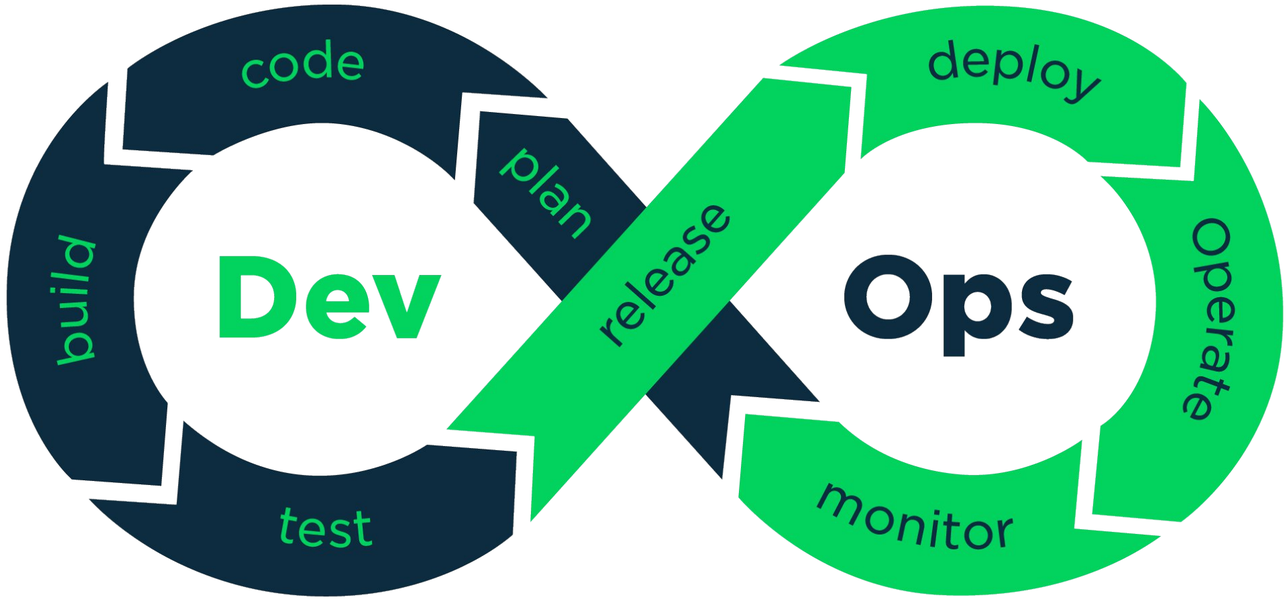
CI et CD

CI - Intégration continue

Chaque changement est testé et intégré automatiquement au code commun.

CD - Déploiement continu

Chaque changement validé est livré automatiquement, avec ses propres étapes de validation.



Anatomie d'un pipeline

Stage

Une étape logique du pipeline (test, build, deploy).

Job

Une tâche exécutée au sein d'un stage.

Runner

La machine qui exécute réellement les jobs.

Runners : quel type choisir

Kubernetes	Meilleures performances, scalabilité — le choix par défaut en production.
Docker	Isolation simple et rapide à mettre en place, le plus courant en équipe.
VirtualBox	Utile quand le job a besoin d'un système d'exploitation complet.
Système hôte	Pas d'isolation : à réserver aux petits projets, à éviter en entreprise.

Artifacts, variables et secrets

Artifacts

Les fichiers produits par un job (binaire, rapport de tests) transmis au job suivant.

Variables & secrets

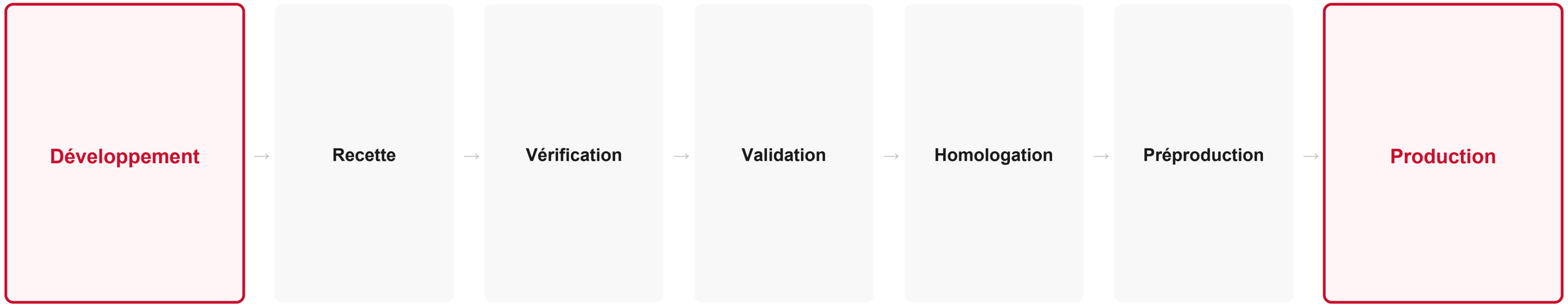
Configuration et données sensibles (clés API, mots de passe) injectées sans jamais apparaître dans le code.

SÉANCE 3 — CI/CD

La gestion des secrets sur GitLab

Variables CI/CD masquées	Déclarées dans les paramètres du projet, jamais dans le `.gitlab-ci.yml` — masquées dans les logs.
Variables protégées	Réservées aux branches et tags protégés — un secret de production n'est jamais exposé à une branche de développement.
Secret detection (scan)	Un job de pipeline dédié scanne chaque commit à la recherche de clés ou mots de passe oubliés dans le code.
Pre-commit hooks	Un contrôle local, avant même le commit, qui bloque l'ajout d'un secret détecté — plus rapide qu'attendre le pipeline.
Rotation régulière	Un secret qui ne change jamais est un secret qui, une fois fuité, reste exploitable indéfiniment.

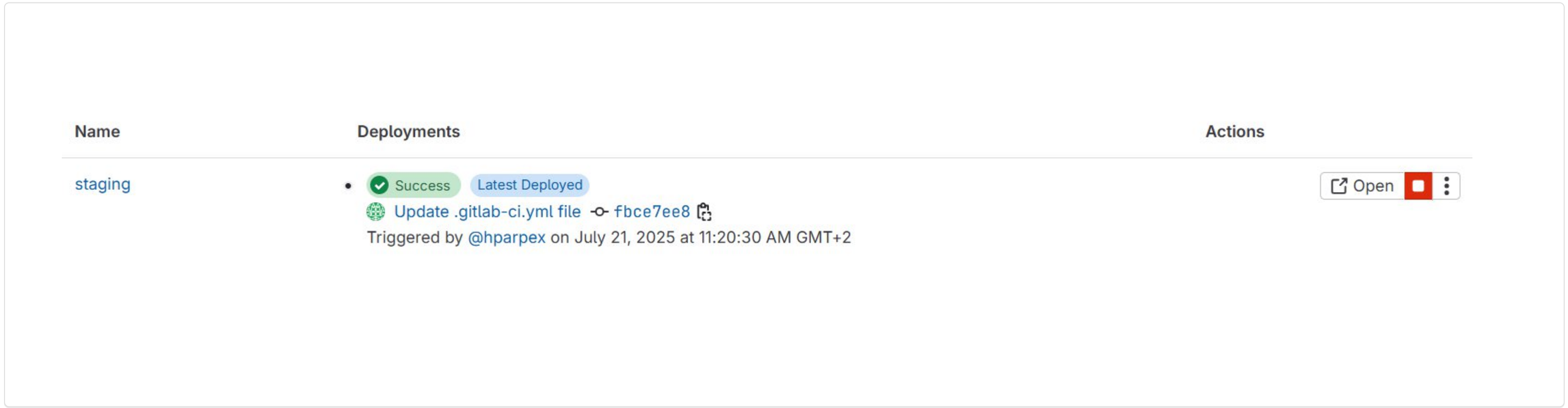
Environnements de déploiement



Chaque environnement se rapproche progressivement des conditions réelles. Seuls le **développement** et la **production** sont incontournables. Les étapes intermédiaires varient selon le projet et son niveau d'exigence.

Un déploiement, vu depuis GitLab

GitLab garde la trace de chaque déploiement par environnement : quand, par qui, et son état.

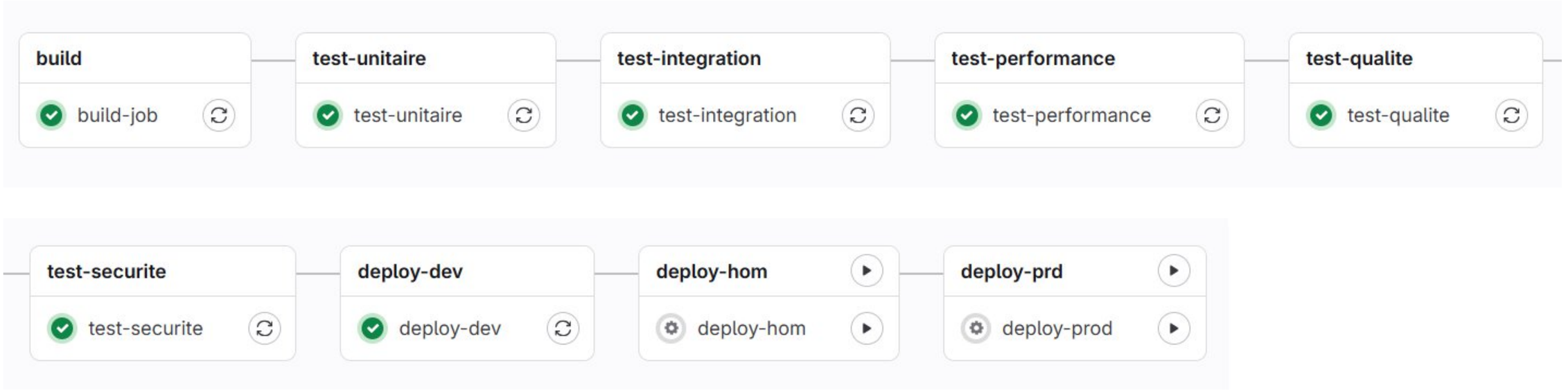


SÉANCE 3 — CI/CD

GitLab CI

Le pipeline est décrit dans un unique fichier `.gitlab-ci.yml`, versionné avec le code, exécuté automatiquement à chaque push.

EXEMPLE D'EXÉCUTION



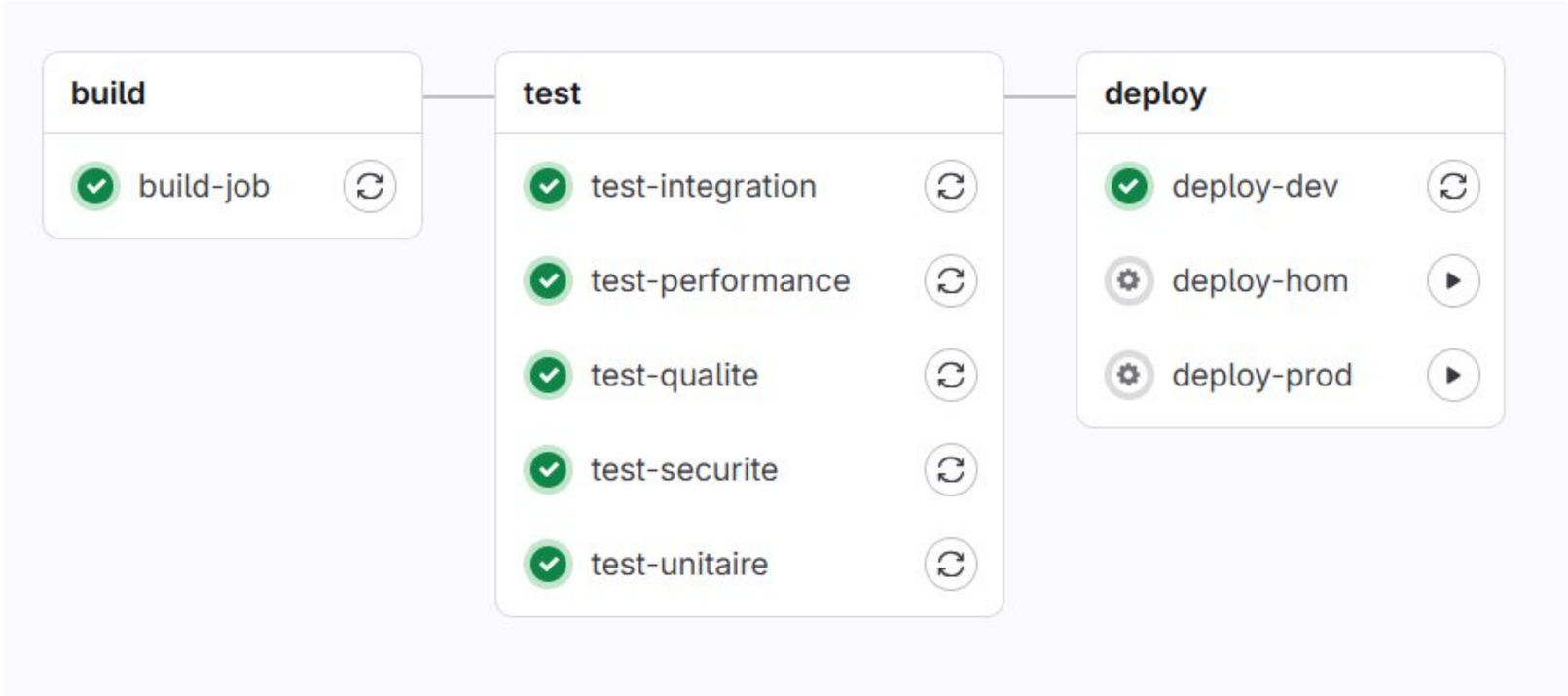
Exemple : .gitlab-ci.yml

```
stages: [test, build, deploy]
test:
stage: test
script: npm test
build:
stage: build
script: docker build -t taskflow-api .
deploy:
stage: deploy
script: kubectl apply -f k8s/
only: [main]
```

SÉANCE 3 — CI/CD

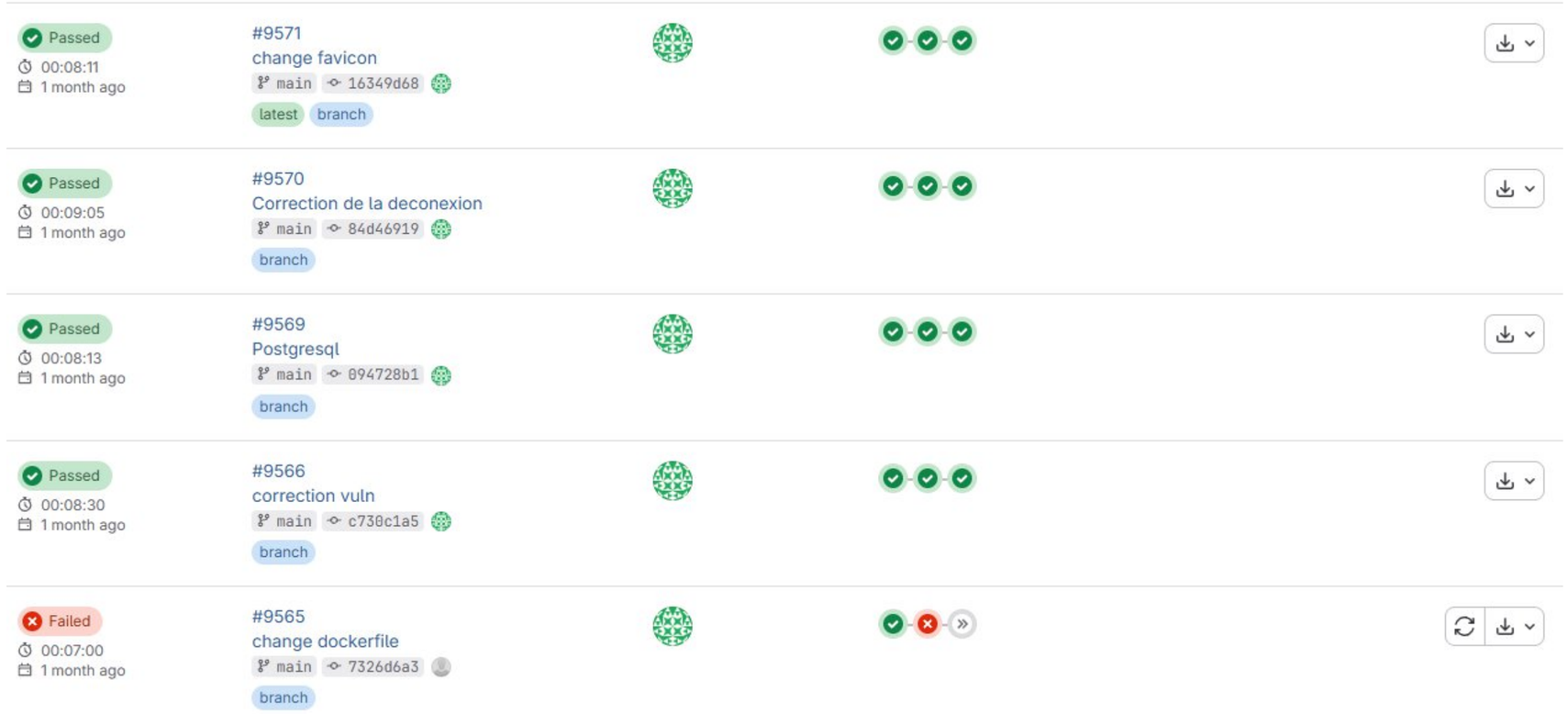
Stages séquentiels, jobs parallèles

Les stages s'exécutent l'un après l'autre — mais à l'intérieur d'un même stage, les jobs tournent en parallèle si des runners sont disponibles.



Ce que ce pipeline produit dans GitLab

Les stages `build`, `test` et `deploy` du `.gitlab-ci.yml` deviennent cette vue, groupée par étape.



SÉANCE 3 — CI/CD

GitHub Actions

L'équivalent chez GitHub : des `workflows` déclenchés par des événements (push, pull request), décrits en YAML dans `.github/workflows/`.

Exemple : workflow GitHub Actions

```
on: push
jobs:
build-and-test:
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v4
- run: npm install
- run: npm test
```

Stratégies de déploiement automatique

Rolling update

Remplacement progressif des instances, sans coupure de service.

Blue-green

Deux environnements identiques, on bascule le trafic d'un coup.

Canary

La nouvelle version reçoit d'abord un petit pourcentage du trafic.

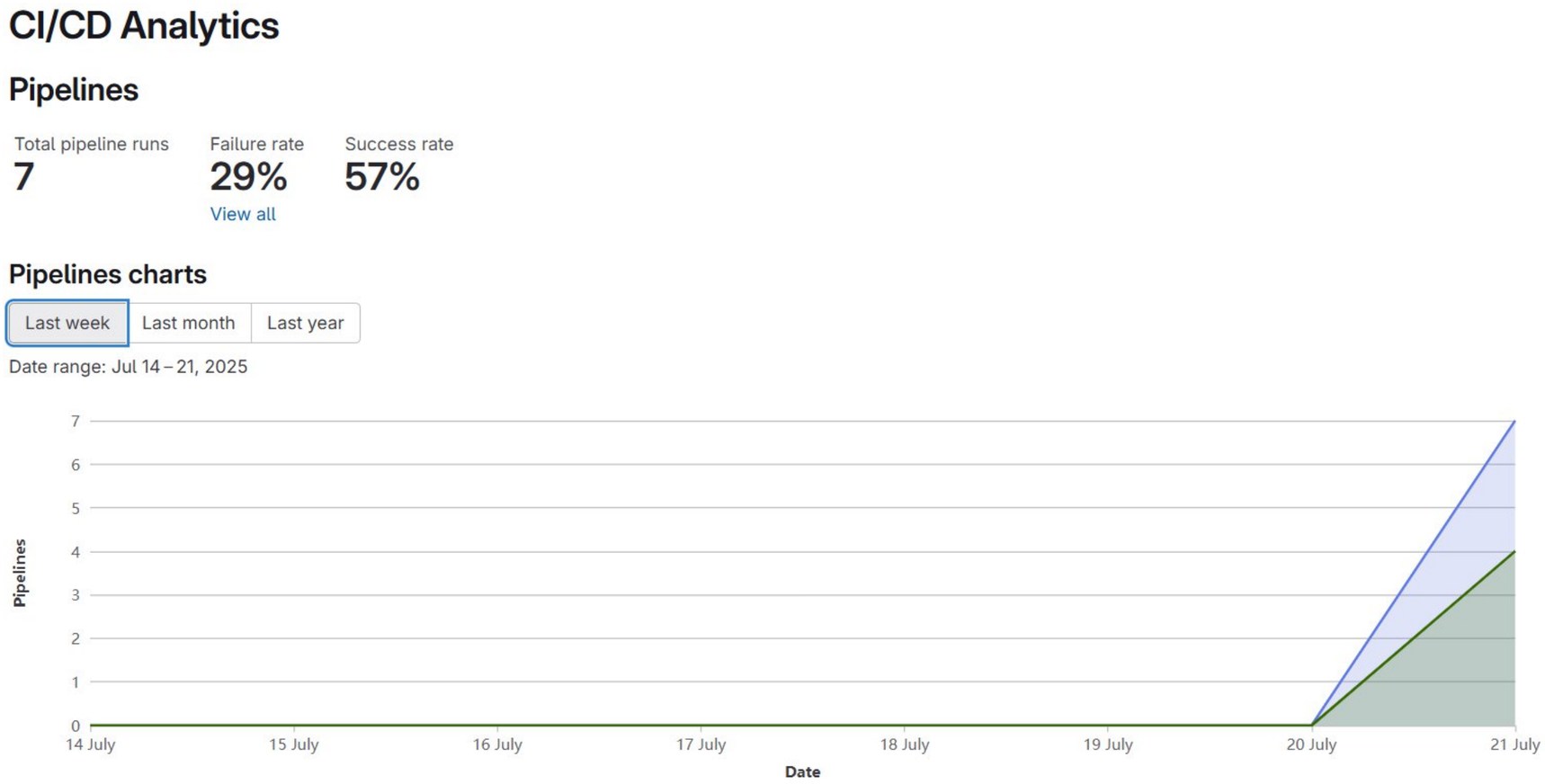
Bonnes pratiques GitLab CI

`cache`	Garde des fichiers d'un job à l'autre, sans les reconstruire.
`artifacts`	Sauvegarde les résultats à conserver : rapports, binaires.
`needs`	Gère les dépendances entre jobs, au-delà de l'ordre des stages.
`tags`	Sélectionne le bon type de runner pour un job.
`timeout`	Définit la durée maximale d'exécution d'un job.
`retry`	Relance automatiquement un job qui a échoué.

SÉANCE 3 — CI/CD

Suivre la santé de ses pipelines

Taux de réussite, durée d'exécution, nombre de commits : GitLab expose ces indicateurs pour repérer un pipeline qui se dégrade.



FIL ROUGE — TASKFLOW

Le pipeline de TaskFlow produit maintenant automatiquement un artefact prêt à déployer.

Prochaine étape : exécuter cet artefact de la même façon partout, avec Docker et Kubernetes.

SÉANCE 4 · 1H30

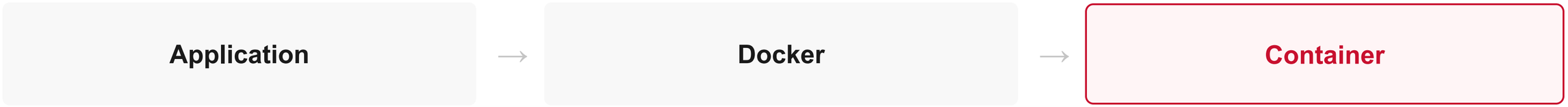
Conteneurisation et orchestration : Docker → Kubernetes

FIL ROUGE — TASKFLOW

Le pipeline produit un build. Comment l'exécuter partout, à l'identique ?

L'API de TaskFlow tourne parfaitement sur la machine d'Alice… mais pas toujours sur celle de Bob, ni sur le serveur de production.

Pourquoi les containers ?



Le container embarque le code et tout son environnement : « ça marche sur ma machine » devient « ça marche partout ».

Image, container, registry

Image

Un modèle figé : code, dépendances, configuration.

Container

Une instance en cours d'exécution de cette image.

Registry

L'entrepôt où les images sont stockées et partagées.

Le Dockerfile : anatomie

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm ci --production
COPY . .
CMD ["node", "server.js"]
```

Construire et lancer une image

```
$ docker build -t taskflow-api .
$ docker run -p 3000:3000 taskflow-api
$ docker push registry.taskflow.io/taskflow-api
```

Pourquoi l'optimisation d'image compte

Vitesse

Une image légère se télécharge et démarre plus vite — crucial pour le scaling.

Sécurité

Moins de paquets installés, moins de surface d'attaque.

Coût

Stockage et bande passante du registry facturés au volume.

Bonnes pratiques d'optimisation

1	**Multi-stage builds**	On compile dans une étape, on ne copie que le résultat dans l'image finale.
2	**Images de base légères**	`alpine` ou `distroless` plutôt qu'une image système complète.
3	**.dockerignore**	Exclure `node_modules/`, `.git` du contexte de build.
4	**Cache des layers**	Ordonner les instructions du moins au plus changeant pour réutiliser le cache.

Avant / après : un Dockerfile optimisé

Naïf — environ 1,1 Go

```
FROM node:20
COPY . .
RUN npm install
CMD ["node", "server.js"]
```

Optimisé (multi-stage) — environ 120 Mo

```
FROM node:20 AS build
COPY . .
RUN npm ci && npm run build
FROM node:20-alpine
COPY --from=build /app/dist ./
CMD ["node", "server.js"]
```

« J'en ai 50, maintenant. »

TaskFlow devient populaire. Il faut désormais plusieurs containers, sur plusieurs machines, qui doivent redémarrer seuls en cas de panne et s'adapter au trafic.

Qui redémarre un container qui plante ? Qui répartit le trafic entre eux ? Qui ajoute des instances quand il y a plus de demande ?

→ Il faut un orchestrateur : Kubernetes.

SÉANCE 4 — DOCKER → KUBERNETES

Pourquoi Kubernetes ?

Kubernetes orchestre des containers : il décide où les faire tourner, les surveille, les remplace s'ils tombent, et les fait évoluer en nombre selon la charge.

L'OBJECTIF : RESTER EN 1H30 SUR L'INTUITION, PAS L'ADMINISTRATION

Cluster et node

Cluster

L'ensemble des machines qui forment l'environnement Kubernetes.

Node

Une machine individuelle du cluster, qui exécute les containers.

Le manifest : décrire l'état voulu

On ne dit jamais à Kubernetes quoi faire étape par étape. On décrit l'état voulu dans un fichier YAML — le manifest — et Kubernetes travaille en permanence à faire correspondre l'état réel à cette description.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
name: taskflow-api
spec:
replicas: 3
template:
spec:
containers:
- name: api
image: taskflow-api:1.4.0
```

apiVersion

La version de l'API Kubernetes utilisée pour cette ressource.

kind

Le type de ressource : Deployment, Service, Pod...

metadata

Son identité : nom, namespace, labels.

spec

L'état désiré : réplicas, image, configuration.

Le Pod : la plus petite unité

Un Pod encapsule un ou plusieurs containers qui doivent toujours tourner ensemble, sur la même machine, avec le même réseau. C'est l'unité que Kubernetes déploie et surveille.

Le cycle de vie d'un Pod

Pending → **Running** → **Succeeded** | **Failed**

Liveness probe

Vérifie que le container est toujours vivant. S'il ne répond plus, Kubernetes le redémarre.

Readiness probe

Vérifie que le container est prêt à recevoir du trafic. Sinon, le Service ne lui envoie rien.

Deployment : gérer les réplicas

Un Deployment décrit combien de copies (réplicas) d'un Pod doivent tourner, et gère les mises à jour progressives sans interruption de service.



Mise à jour vers v1.5.0 — un Pod remplacé à la fois

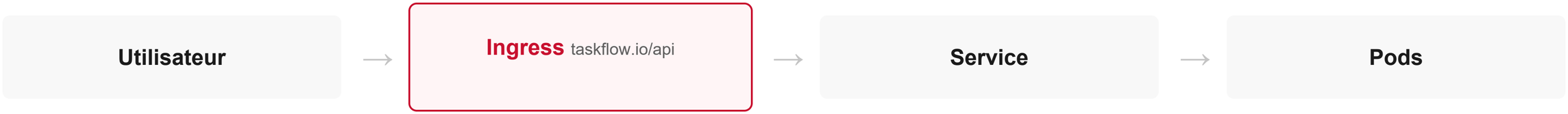


Service : exposer les Pods

Les Pods vont et viennent, avec des adresses changeantes. Un Service leur donne une adresse stable et répartit le trafic entre les réplicas disponibles.

Router le trafic vers l'application

`ClusterIP`	Adresse interne au cluster seulement — pour la communication entre services.
`NodePort`	Ouvre un port fixe sur chaque node — simple, mais peu utilisé en production.
`LoadBalancer`	Provisionne une adresse IP publique chez le fournisseur cloud.



L'Ingress route selon le nom de domaine ou le chemin d'URL, vers le bon Service — un seul point d'entrée pour plusieurs applications.

Namespace, ConfigMap et Secret

Namespace

Isole des groupes de ressources — un par environnement, par exemple.

ConfigMap / Secret

Séparent la configuration (ConfigMap) et les données sensibles (Secret) du code de l'application.

Une base, des overlays par environnement

TaskFlow tourne en développement, staging et production avec des besoins différents (réplicas, ressources). Kustomize évite de dupliquer les manifests pour chaque environnement.

```
base/
deployment.yaml
service.yaml
kustomization.yaml
overlays/
dev/kustomization.yaml
staging/kustomization.yaml
prod/kustomization.yaml
```

Ne jamais dupliquer les manifests

La base reste unique ; chaque overlay ne contient que ses différences.

Patcher, pas réécrire

Un overlay ajuste replicas, ressources ou variables — jamais toute la ressource.

Un overlay par environnement

dev, staging, prod — chacun avec son propre nombre de réplicas et ses ressources.

Scaling et rolling update

Scaling

Ajouter ou retirer des réplicas selon la charge, manuellement ou automatiquement.

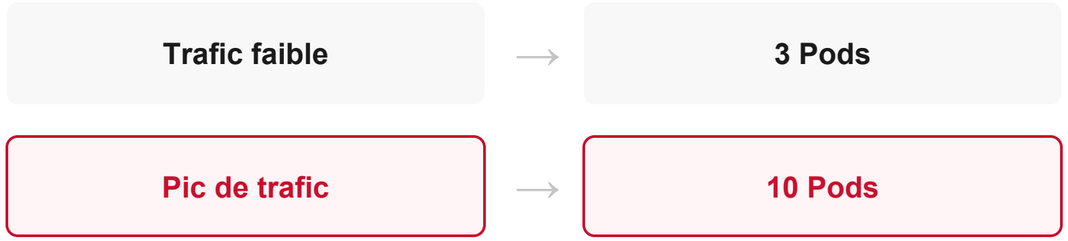
Rolling update

Remplacer les anciens Pods par la nouvelle version, un par un, sans coupure.

Scalabilité automatique : le HPA

Le **Horizontal Pod Autoscaler** observe une métrique (le plus souvent le CPU) et ajuste automatiquement le nombre de réplicas entre un minimum et un maximum.

```
kind: HorizontalPodAutoscaler
spec:
minReplicas: 3
maxReplicas: 10
metrics:
- type: Resource
resource:
name: cpu
target:
averageUtilization: 70
```



Surveiller un cluster Kubernetes

Monitoring Kubernetes

metrics-server

Collecte CPU et mémoire des Pods, consultable via `kubectl top`.

Prometheus

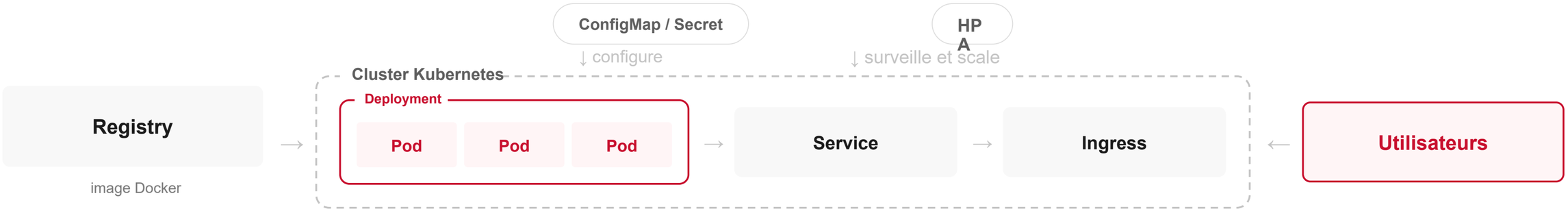
Collecte ces métriques dans le temps, pour repérer des tendances.

Grafana

Affiche ces métriques en tableaux de bord visuels et déclenche des alertes.

Les probes de liveness et readiness alimentent aussi l'état affiché par `kubectl get pods` — la première brique du monitoring.

Architecture générale



Exemple : deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
name: taskflow-api
spec:
replicas: 3
template:
spec:
containers:
- name: api
image: registry.taskflow.io/taskflow-api:1.4.0
```

Commandes kubectl essentielles

```
$ kubectl apply -f deployment.yaml
$ kubectl get pods
$ kubectl describe pod taskflow-api-7d9f
$ kubectl logs taskflow-api-7d9f
$ kubectl delete pod taskflow-api-7d9f
```

Piloter et débugger un déploiement

`kubectl scale --replicas=5`	Change manuellement le nombre de réplicas d'un Deployment.
`kubectl rollout status`	Suit la progression d'une mise à jour en cours.
`kubectl rollout undo`	Revient à la version précédente en cas de problème.
`kubectl exec -it ... -- sh`	Ouvre un terminal à l'intérieur d'un container, pour investiguer.
`kubectl top pods`	Affiche la consommation CPU et mémoire en temps réel.

RÉCAPITULATIF

FIL ROUGE — TASKFLOW

Du ZIP au cluster qui scale

1. Le code est versionné et travaillé en équipe avec Git
2. Chaque push est testé et construit automatiquement par un pipeline CI/CD
3. L'application est conteneurisée dans une image Docker optimisée
4. Kubernetes l'exécute, la surveille et la fait évoluer selon la charge

CONCLUSION

Questions

Modalités d'évaluation, TP et évaluation sur table — les prochaines séances mettront ces notions en pratique.